

Symbolic and Materialization Layers for Computational Materials Science: Principles and First-Demonstration Plan

G. Barbalinardo
University of California, Davis

May 11, 2026

Abstract

This document records the design of a typed symbolic/materialization framework for computational materials science workflows, organized around a small set of principles. Workflows are directed acyclic graphs of *symbolic physics states* connected by *symbolic operations*, and concrete numerical results are *materializations*—typed projections of symbolic states into a discretization scheme. The symbolic root is the *Born–Oppenheimer potential* of a material, expanded as a Taylor series in atomic displacements; force constants of every order are derived states obtained by extracting derivatives of this root and truncating the expansion at a chosen order. Every state carries a provenance chain that records the operations along its path; every materialization carries a discretization scheme. Approximation and discretization errors live in disjoint type universes and compose along graph edges; the machinery for tracking error formulas symbolically and composing them across the graph is designed but its full implementation is deferred to Phase 2. As a first demonstration, three established lattice thermal-transport codes (kaldo, phono3py, ShengBTE) are reframed as distinct materializations of a single symbolic DAG, with cross-code reconciliation at canonical intermediate states as the Phase 1 deliverable. Phase 1 stubs the upstream operations that produce force constants from the potential—declaring the relevant symbolic types and provenance labels but not modeling them in detail—so that Phase 2 can elaborate the upstream without restructuring the framework. The document is intended as the working architectural reference for the `openmaterials-ai` project; it covers each architectural decision, the alternatives considered, and what we deferred.

Contents

1	Principles	2
2	Background and Motivation	3
3	Architectural Principle: Two Worlds, One Bridge	4
3.1	The full chain: two layers plus the external codes	5
4	Formal Definitions	6
4.1	Symbolic state	6
4.2	Symbolic operation	8
4.3	Symbolic workflow	9
4.4	Discretization scheme	9
4.5	Materialization	9
4.6	Cross-adaptor comparison: <code>compare</code>	9

4.7	HiddenState discipline and gauge actions	10
4.8	Materialization functor	11
4.9	Total error	11
5	Lean-compatibility disciplines	11
6	First Demonstration: Lattice Thermal Transport	12
6.1	The symbolic DAG	12
6.2	Materialization functors (codes)	13
6.3	Verification on silicon (Tersoff potential)	14
6.4	Phase 1 scientific capabilities	15
7	Implementation layout	15
8	Open Questions and Deferred Decisions	16
8.1	Algebra of error composition (deferred to Phase 2)	16
8.2	When to lift to Lean	16
8.3	Provenance equivalence	16
8.4	Sprint 1 scope	17
8.5	Implementation language	17
9	Outlook	17

1 Principles

The symbolic layer is built on a small set of commitments. They are stated here without their alternatives or justifications; supporting definitions follow in later sections.

1. **Two worlds, one bridge.** The symbolic layer keeps the symbolic physics world (typed states, symbolic operations, approximation errors) strictly separate from the numeric world (discretized arrays, computed values, discretization errors). They are connected by exactly one bridge: the materialization functor.
2. **Symbolic states are unit-free.** Dimensionful quantities (frequency, energy, length, time) exist at the symbolic layer, but specific unit choices (THz vs eV vs rad/s, Å vs Bohr) do not. Unit choice belongs to the materialization and is declared by the adapter that produced it; the framework normalizes to a canonical unit before any cross-materialization comparison.
3. **The atom is a symbolic operation.** The atomic unit of meaning is a typed transformation between symbolic states, not an instruction in an input file or an algorithm. Every operation carries a *symbolic formula*—a sympy expression (closed-form output) or sympy.Eq (implicit equation defining the output)—stating what it computes. The formula is the symbolic claim, machine-readable and Lean-projectable, against which adapter conformance can be checked.
4. **States are typed witnesses; nodes are Observables or HiddenStates.** A state is the typed claim that a symbolic physics object exists with a particular history. The symbolic layer splits states into two structural kinds: *Observables* (gauge-invariant, first-class, required to agree across adapters) and *HiddenStates* (gauge-dependent intermediate scaffolding whose per-element values reflect basis / phase / BZ-summation choices the framework does not pin down). Each state declares fields, and each field has an index

signature (e.g. $\omega(\mathbf{q}, \nu)$ has indices $(\mathbf{q}, \nu)$; $\kappa^{\alpha\beta}$ has (α, β)). Numerical content lives in the materialization, separate from the witness.

5. **Workflows are directed acyclic graphs.** A workflow is a graph of states connected by operations, not a linear sequence. Cross-code comparison reduces to graph alignment over the same symbolic template.
6. **Identity is type plus provenance.** A symbolic state is identified by its physics type and the ordered chain of operations that produced it. Two states with the same type but different histories are different states.
7. **Operation identity is parameterized.** An operation is identified by its name together with the parameters that affect physics. Different physics-affecting parameter values produce different operations and therefore different output states. Function-family parameterizations (e.g., Gaussian broadening by standard deviation vs. by half-width) are canonicalized at the symbolic layer to a single choice; adapters translate to their internal convention.
8. **Discretization belongs to materialization.** Mesh densities, supercell sizes, displacement steps, and broadenings live on the materialization, not on the symbolic state. Convergence is a relationship between a symbolic state and a family of its materializations.
9. **Cross-code agreement is required at Observable nodes only.** Two adapters' materializations of an Observable must agree (after unit and convention conversion) to within the observable's declared tolerance. Materializations of a HiddenState are not cross-code comparable per-element; the symbolic layer's `compare` returns the status `NOT_COMPARABLE` and reports residuals as diagnostic information only. To make a HiddenState cross-code comparable, contract it into an Observable (e.g., per-mode $\Gamma_{q\nu}$ is a HiddenState; $\sum_{q\nu} \Gamma$ contracted via the BZ sum is an Observable that does agree).
10. **Errors are designed-for.** Approximation errors live on operations; discretization errors live on materializations. The type system reserves slots for both; symbolic composition of error formulas is a Phase 2 capability.
11. **The symbolic root is the potential.** Every materialization in the framework ultimately traces back to the Born–Oppenheimer potential of the material. Force constants of every order are derived states obtained by extracting derivatives of the potential and truncating the Taylor expansion. The upstream is stubbed in Phase 1.
12. **Sources are produced by nullary operations.** Source states (the potential, but also temperature, lattice constant, etc.) are the outputs of nullary `provide_*` operations rather than nodes with empty provenance. This unifies the DAG: every node is the output of some operation, and the symbolic layer has no special “input” category. The materialization of a source state carries the externally supplied value (e.g., $T = 300\text{ K}$); the operation is just a structural tag.
13. **Lean-compatible by structure.** The symbolic layer is implemented in Python but designed so that its types transliterate cleanly into Lean. Three disciplines protect this option: closed unions for physics types, no Python-encoded physics invariants, operation parameters always recorded in output state metadata.

2 Background and Motivation

Modern computational materials science has many codes, many workflows, and many opinions about how to compose them. Despite a generation of effort on workflow managers (AiiDA, ATOMATE, FIREWORKS, ATOMATE2, AFLOW), one quality is still missing: a workflow should expose, as typed and composable data, *the physics it computes*.

Today the standard unit of shared meaning between codes is the *input file*: an ordered sequence of instructions that, together with a particular code’s internal logic, produces an output. This unit is opaque to two questions:

- **Given a result, what physical assumptions and approximations does it embed?** An input file declares parameters but not their meaning. A k-mesh is just a triple of integers; the relationship between that mesh and the convergence of the integrated observable is not part of the file.
- **Given two results from two codes, where exactly do they diverge?** Identical force constants run through kaldo and through ShengBTE typically yield slightly different thermal conductivities. The difference is not localized to a particular operation, even though—in principle—it is the result of distinct computational choices at specific steps.

A previous attempt of ours, `MaterialsCodeGraph` (MCG), addressed parts of this gap by treating each instruction in an input file as the atomic unit of a knowledge graph and tracking inputs, parameters, and outputs at that level. MCG’s tool-encapsulation principle (all tool knowledge in YAML/Markdown configs, all code generic over tool) was the right design at that level of abstraction. But the level of abstraction itself is too low: an instruction in an input file is a syntactic unit, not a semantic one. It tells you what a code is told to do, not what *physics* the code thereby computes. Two codes that compute the phonon dispersion via diagonalization of the dynamical matrix express that operation as different instructions; yet they perform the same physics. A graph indexed by instructions sees them as two unrelated tasks.

The symbolic layer proposed here *starts from the physics* and treats input files as derived artifacts. The atomic unit is no longer an instruction; it is a *symbolic operation* between typed physics states. The states are the load-bearing entities: a state is what changes when an operation is applied, and a workflow is the graph that records those state transitions. Concrete numerical computation is then a separate, typed projection of the symbolic workflow into a discretization scheme.

This is a deliberate departure from the input-file paradigm. We argue below that it pays for the extra abstraction with three concrete returns: cross-code reconciliation at canonical intermediate states, derived (rather than empirical) error bounds on computed observables, and a clean foundation for future formal verification.

3 Architectural Principle: Two Worlds, One Bridge

The single most important design decision is a strict separation between two type universes:

- **The symbolic world** holds typed physics states (Hamiltonians, dispersion relations, scattering rates, distribution functions, partition functions) and symbolic operations between them. Errors here are *approximation errors*—the $O(\cdot)$ terms introduced by truncating perturbation series, linearizing equations, or imposing physical assumptions such as the Born–Oppenheimer or harmonic approximation. The symbolic world contains no numerical content. Two symbolic states that differ only in their numerical estimates are the same symbolic state; two symbolic states that differ in the chain of approximations leading to them are different symbolic states.

- **The numeric world** holds discretized realizations of symbolic states: arrays sampled on finite grids, mode-resolved quantities at finite k-meshes, distributions evaluated at finite temperature. Errors here are *discretization errors*—the $O((1/N)^k)$ or $O(h^k)$ terms introduced by sampling a continuum quantity on a finite scheme. The numeric world contains no symbolic content; it does not know what theory the numbers are estimates of.

The two worlds are connected by exactly one bridge, the *materialization functor*, which takes (i) a symbolic state and (ii) a discretization scheme and produces a numeric materialization. Approximation errors compose in the symbolic world along provenance chains; discretization errors compose in the numeric world along execution graphs; the two combine only at materialization time, where the total error of a numerical result decomposes into its symbolic and numeric contributions.

This separation is load-bearing. It is also somewhat unusual. Most computational physics codes mix the two worlds: a “Hamiltonian” object in such a code is sometimes a typed mathematical operator and sometimes a 4-D `numpy` array. The symbolic layer refuses to do this. If you are holding a Hamiltonian, you know which world you are in and you cannot accidentally mix the two. This refusal is what makes errors composable, theorems statable, and cross-code comparisons well-defined.

3.1 The full chain: two layers plus the external codes

In implementation terms, the project has *exactly two layers* under its own control plus the external physics codes that produce numerical data. Nothing else is a layer: tests, visualization, and experiment scripts are supporting infrastructure.

Layer 1 — symbolic. Types, declarations, formulas, gauges. Machinery in `omai.symbolic` (the `State` and `Operation` types, `Field`, `Dimension`, `GaugeAction`, `SymmetryGroup`, `validate_dag`); domain instances in `omai.thermal_transport.symbolic` (the 14-node, 13-edge DAG with sympy formulas on each edge, gauge actions). No values, no units, no per-code knowledge.

Layer 2 — materialization. Per-code declarations plus the runtime that operates on actual code outputs. Machinery in `omai.materialization` (the `StateAdapterSpec` and `OperationAdapterSpec` types, `Unit` and unit conversions, the `Materialization` runtime data class, `compare()` with its five-status verdicts); domain instances in `omai.thermal_transport.materialized` (one file per code: `kaldo.py`, `phono3py.py`).

External codes. `kaldo`, `phono3py`, `ShengBTE`: do the actual physics computation in their own native code, output arrays in their native units and conventions.

The chain in motion. A typical cross-code verification flows through these layers like this:

1. User invokes the codes externally (e.g. `kaldo` and `phono3py`), which produce raw output arrays in their own formats.
2. User wraps each array via `materialize(spec, field_name, array)`: this validates the field exists on the symbolic state and produces a `Materialization` object tagged with the right adapter spec.
3. User calls `compare(m_a, m_b)`. `compare` consults the symbolic layer for the cross-code factor (units + conventions), applies it, optionally contracts, and consults the symbolic layer again for the state’s gauge kind to decide whether a PASS/FAIL verdict is even meaningful or the comparison is NOT_COMPARABLE.

4. User gets a `ComparisonResult` with status, residuals, and the applied factor.

No additional layers, no other indirection. The two substrate-side layers expose their content in matched pairs (machinery in `omai/<layer>/`, domain instances in `omai/<domain>/<layer>/`), and the codes attach at the edge.

4 Formal Definitions

We collect the formal definitions implied by the decisions above.

4.1 Symbolic state

A symbolic state s is unit-free: its physics type carries the dimension of any quantity it represents (a frequency, an energy density, a tensor of given rank), but no unit choice. Numerical content and units appear only at materialization time.

A symbolic state s is a tuple (κ, τ, π, F) where:

- $\kappa \in \{\text{OBSERVABLE}, \text{HIDDENSTATE}\}$ is the *gauge-invariance kind*: Observables are gauge-invariant and cross-code comparable; HiddenStates carry gauge freedom (eigenvector phase, BZ-summation choice, degenerate-subspace rotation, ...) that the symbolic layer doesn't pin down, so their per-element values differ across adapters by design;
- τ is a *physics type* drawn from a finite registry. Derived types in the thermal-transport scope include `ForceConstants[order= $n$ ]`, `DynamicalMatrix`, `Frequency`, `Eigenvectors`, `GroupVelocity`, `HeatCapacity`, `Linewidth`, `MeanFreeDisplacement`, `ThermalConductivity`; sources include `Potential`, `Temperature`. Some types are further parameterized by upstream operation choices (e.g., `ThermalConductivity[bte_solver= rta]` is a HiddenState since RTA's $1/\Gamma$ non-linearity breaks gauge invariance, whereas `ThermalConductivity[bte_solver=direct]` is an Observable because the full LBTE preserves it);
- π is the *provenance*: an ordered list $[\mathcal{O}_1, \mathcal{O}_2, \dots, \mathcal{O}_n]$ of symbolic operations whose composition produced s . Provenance is never empty—source states have a single nullary operation (`provide_potential`, `provide_temperature`, ...);
- $F = (f_1, f_2, \dots)$ is the state's tuple of *fields*. Each field has a name, a dimension (frequency, energy, length, ...), and an index signature (e.g., ω has indices $(\mathbf{q}, \nu)$; v has $(\alpha, \mathbf{q}, \nu)$; κ has (α, β)).

Two symbolic states are equal iff their names match. The composed approximation error on s (a Phase 2 capability) would be

$$\epsilon_{\text{approx}}(s) = \text{compose}(\epsilon_{\mathcal{O}_1}, \epsilon_{\mathcal{O}_2}, \dots, \epsilon_{\mathcal{O}_n}). \quad (1)$$

Source versus derived states. Symbolic states fall into two kinds, both produced by some operation:

- *Source states* are the outputs of *nullary* operations (`provide_potential`, `provide_temperature`, ...): operations with no symbolic-state inputs. They are empirical inputs (lattice constant, temperature, pressure, applied field), the symbolic `Potential` itself (Phase 1 stubs the upstream of the potential), or measured observables (a thermal conductivity reported from a TDTR experiment, a Raman peak). The materialization of a source state carries the externally supplied value (e.g., 5.43 Å for the lattice constant of silicon, or $T = 300$ K); its discretization scheme is trivial (a singleton choice rather than a refinable mesh) but its error is still meaningful (experimental uncertainty, or definitional precision).

- *Derived states* are the outputs of operations that consume one or more upstream states. Examples: `ForceConstants[order=n]` (derived from `Potential` via `compute_force_constants[order=n]`, currently stubbed), `Frequency` and `Eigenvectors` (multi-output of `compute_dispersion`), `Linewidth`, `MeanFreeDisplacement`, the computed κ .

Both kinds follow the same machinery for materialization. The unification of source and derived states under the same “output of an operation” rule answers several otherwise awkward cases: empirical inputs (“where does the lattice constant live?”), experimental observables (“how does a measured $\kappa(T)$ enter the framework?”), and the unmodeled symbolic root in Phase 1 (“what is the silicon potential, before we differentiate it?”). All three are materializations of source states whose provenance is a single nullary operation; the second is what makes the unified theory–experiment comparison framework native rather than bolted on; the third is how Phase 1 stubs the upstream of `Potential` without yet modeling it in detail.

What “witness-as-state” means, concretely. The phrase *witness-as-state* comes from the Curry–Howard correspondence in type theory: a typed value is read as evidence (a “witness”) for a proposition. We borrow the intuition without borrowing the formal proof obligations. In this framework a state is not the data it contains; it is the *claim* that something exists, expressed in a typed form rich enough to carry meaning.

Concretely, three layers are conflated in most computational physics codes; the symbolic layer keeps them apart:

1. **The proposition** – “the phonon dispersion of crystalline silicon under the harmonic approximation, derived by truncating the Born–Oppenheimer Taylor expansion at second order, exists as a function $\omega(\mathbf{q}, \nu)$ from the Brillouin zone to positive reals.”
2. **The witness** – a typed object $\mathbf{s} = (\text{Dispersion}, \pi)$ that records precisely this claim. The physics type `Dispersion` encodes the kind of object being asserted to exist; the provenance π records the chain of operations (extraction, truncation, Fourier transform, eigendecomposition) that produced it. The witness carries no numerical content; it is the assertion that such an object exists with this history.
3. **The materialization** – a tuple $(s, \Sigma, x, \epsilon_{\text{disc}})$ in which x is a concrete 4-D `numpy` array of frequencies sampled on a chosen k-mesh, Σ is the discretization scheme, and ϵ_{disc} is the associated discretization error. The materialization is the numerical evidence that the witness asserts to exist.

A code that conflates these layers (treats the dispersion *as* the array) cannot distinguish “the same physics computed two different ways” from “different physics computed the same way.” A framework that keeps them apart can: two witnesses with identical types but different provenance are different states, regardless of whether their materializations happen to have similar numerical content; one witness with two materializations on different k-meshes is the same state, regardless of how different the arrays look numerically.

This is the load-bearing distinction. It is also the answer to several otherwise hard questions. “What is the dispersion of silicon?” is the question the witness answers. “What does the dispersion of silicon look like on a $14 \times 14 \times 14$ mesh in kaldo” is the question its materialization answers. “Are these two computed dispersions the same?” resolves into two sub-questions—are the witnesses equal (a question about provenance), and are the materializations consistent within their discretization errors (a question about numerics)—each answerable without confusion with the other.

Read in this light: the symbolic states described elsewhere in this document are all witnesses: they assert the existence of a particular physics object derived in a particular way. The numerical content, when present, lives in the materialization, separated from the witness by the symbolic/materialized boundary. Operations between symbolic states are operations between witnesses, with no numerics involved; they are essentially structural moves on a graph of typed assertions.

4.2 Symbolic operation

A symbolic operation $\mathcal{O}$ is a typed map between symbolic states,

$$\mathcal{O} : \tau_{\text{in}_1} \times \tau_{\text{in}_2} \times \cdots \rightarrow (\tau_{\text{out}_1}, \dots, \tau_{\text{out}_m}), \quad (2)$$

equipped with three pieces of metadata:

- a *symbolic formula* $\mathcal{F}_{\mathcal{O}}$ stating what the operation computes, encoded as a sympy expression for closed-form ops or a sympy.Eq for implicit ones (eigenvalue equations, linear systems). *Every* operation carries a formula—source operations have identity formulas (e.g., `provide_temperature: T = T_provided`);
- an *approximation error formula* $\epsilon_{\mathcal{O}}$, a symbolic expression in physical small parameters (e.g., λ for the strength of a perturbation, T/T^* for a Sommerfeld expansion). Exact operations have $\epsilon_{\mathcal{O}} = 0$; approximating operations have $\epsilon_{\mathcal{O}} = O(p^k)$ for some parameter p and order k ;
- a possibly empty set of *algorithmic conventions* (per §4.2) and *parameters* (dimensioned but unit-free at the symbolic layer).

Multi-output operations (e.g. `compute_dispersion` producing `Frequency` and `Eigenvectors` together) are first-class. The formula $\mathcal{F}_{\mathcal{O}}$ is the symbolic layer’s machine-readable claim about what the operation produces: adapter conformance can be expressed as “the adapter’s runtime behavior matches $\mathcal{F}_{\mathcal{O}}$ under its declared unit and convention choices,” in principle automatable rather than reverse-engineered from kernels.

Operation identity is parameterized. Several operations in the symbolic layer carry method choices that affect physics: the scattering processes included (3-phonon only, or 3-phonon plus isotopic, or with boundary scattering), the Boltzmann solver (relaxation-time approximation, iterative, direct inversion), the broadening scheme (Gaussian, Lorentzian, adaptive), and so on. The symbolic layer treats these as part of the operation’s *identity*, not as runtime configuration that gets discarded.

Concretely, an operation is identified by the tuple (operation name, parameters affecting physics). Two invocations of `compute_scattering_rates` with different `scattering_processes` are different operations; they appear differently in the provenance, and they produce symbolic states that are formally distinct (different type metadata, different total approximation error). The *name* `compute_scattering_rates` is the same; the *parameterized identity* differs.

This convention has three consequences. First, the vocabulary of operations remains small (one entry per named transformation, not one entry per parameter combination), but the provenance remains expressive (because provenance records the full parameterized identity). Second, output states’ types are mildly dependent: `ScatteringRates` is parameterized by the scattering-processes choice, even though the symbolic layer does not require Python’s type system to enforce this dependence. Third, the symbolic layer is structurally compatible with dependent type theory: when transliterated to a system like Lean, the parameter values lift to type-level indices and the dependence becomes formal.

4.3 Symbolic workflow

A symbolic workflow is a finite directed acyclic graph $W = (V, E)$ where V is a set of symbolic states and E is a set of symbolic operations. The graph is rooted at one or more *source* states (typically empirical inputs such as the crystal structure) and terminates at one or more *observable* states (e.g., the lattice thermal conductivity).

4.4 Discretization scheme

A discretization scheme Σ is a finite tuple of parameters $(N_1, \dots; h_1, \dots)$ specifying how a continuum quantity is sampled. Examples include the k-point mesh density, the supercell size, the finite-difference displacement step, the broadening width, and the integration order. The scheme also carries an error model $\epsilon_\Sigma(N_1, \dots; h_1, \dots)$ giving the asymptotic dependence of the discretization error on these parameters.

4.5 Materialization

A materialization m is a runtime data class wrapping one adapter’s emission of one field on one state:

$$m = (\text{STATEADAPTERSPEC}, \text{field_name}, x)$$

where `STATEADAPTERSPEC` carries the symbolic state s , the adapter name, and the adapter’s declarations (units of each field, convention overrides, notes); `field_name` selects which field of s is materialized; and x is a concrete `numpy.ndarray`. The discretization scheme Σ and discretization error ϵ_{disc} are deferred to Phase 2.

Why the adapter spec is part of the materialization. Different codes express the same physics in different units (kaldo’s linewidth is in angular-frequency THz while phono3py’s is in linear-frequency THz; kaldo’s heat capacity is in J/K, phono3py’s in eV/K) and under different conventions (kaldo’s Γ is the linewidth $2\text{Im}\Sigma$ while phono3py’s is $\text{Im}\Sigma$ directly). A raw array without its adapter spec is unsafe to compare against another. With the spec attached, the symbolic layer computes the cross-adapter conversion factor mechanically and surfaces unit / convention mismatches as type-level declarations rather than as numerical mysteries.

4.6 Cross-adapter comparison: compare

The bridge from spec to verified empirical fact is the function

$$\text{compare}(m_a, m_b; \text{rtol}, \text{atol}, \text{contraction}) \rightarrow \text{COMPARISONRESULT}$$

which takes two materializations of the same state and field, applies the cross-adapter conversion factor derived from their declared units and conventions, optionally contracts both arrays via a user-supplied callable (e.g. `numpy.sum`), and reports residuals together with a *status* drawn from five values:

Expected_Pass the state is an Observable; the arrays agree within (rtol, atol). The symbolic layer’s prediction holds.

Expected_Loose the user predicted disagreement (override) and the arrays disagree. Useful for intermediate contractions of a HiddenState that the user knows is only partially gauge-invariant.

Not_Comparable the state is a HiddenState and no contraction was supplied. The symbolic layer refuses to make a pass/fail verdict; residuals are still computed and returned for diagnostic inspection, but they carry no normative weight.

Unexpected_Fail the state is an Observable but the arrays disagree. *Real anomaly*: either the spec is missing a convention, or the codes genuinely compute different physics, or `rtol` is too strict.

Unexpected_Pass the user predicted disagreement and the arrays agreed. Rare; the per-element protocol may be tighter than declared.

The status taxonomy separates three things that earlier versions of this framework conflated: the symbolic layer’s *prediction* (would these agree?), the empirical *outcome* (did they?), and whether the comparison is even *meaningful* for the given state kind. Only `UNEXPECTED_FAIL` flags an anomaly that needs investigation; `EXPECTED_PASS` and `EXPECTED_LOOSE` are successful predictions; `NOT_COMPARABLE` is a `HiddenState` saying “don’t ask this question.”

4.7 HiddenState discipline and gauge actions

The Observable / `HiddenState` distinction is load-bearing, so the symbolic layer enforces a discipline on how `HiddenStates` are declared. Each `HiddenState` carries three additional fields:

- **gauge_group**: a named identifier for the gauge equivalence acting on this state (e.g. “U(1)_phase_on_eigenve” “bz_summation_permutation”).
- **kind**: one of **scaffolding** (the `HiddenState` is consumed by a downstream operation producing an Observable, so the gauge orbit is eventually summed away) or **approximation** (the `HiddenState` is terminal — an approximation of an Observable that happens to break gauge invariance, with no downstream recovery; `κ[bte_solver=rta]` is the canonical example).
- **gauge_invariant_contractions**: names of Observables in the DAG that capture this state’s gauge-invariant content (empty for **approximation** kind).

A `validate_dag` function walks the node and edge sets at load time and rejects: undeclared gauge groups, invalid kinds, scaffolding `HiddenStates` with empty or non-resolving contractions, approximation `HiddenStates` that incorrectly declare contractions, and edges that reference unknown nodes. The thermal-transport DAG passes this validator; tests check it on every CI run.

Symbolic invariance via GaugeAction

Beyond named declarations, the symbolic layer supports *symbolic proofs* of gauge invariance for the tractable cases. A **GaugeAction** is a sympy-encoded transformation: a pattern to match (e.g., $e_{i,q,\nu}$) plus a transform (e.g., $e^{i\theta_{q,\nu}} \cdot e_{i,q,\nu}$). Given an operation’s sympy formula, the symbolic layer substitutes the gauge action and runs `sympy.simplify` on the difference; if the result is zero, the formula is machine-verified invariant.

Example: applying the U(1) phase $e_{i,q,\nu} \rightarrow e^{i\theta_{q,\nu}} e_{i,q,\nu}$ to

$$v_{q,\nu}^\alpha = \frac{1}{2\omega_{q,\nu}} \sum_{i,j} e_{i,q,\nu}^\dagger \frac{\partial D_{ij}}{\partial q^\alpha} e_{j,q,\nu}$$

gives $e^{-i\theta_{q,\nu}} \cdot e^{i\theta_{q,\nu}} = 1$, and sympy mechanically reduces the difference to zero. This is the symbolic layer’s first machine-verified physics claim — per-mode group velocity is U(1)-phase invariant, by construction of the formula.

What’s tractable today: simple symbolic substitutions (U(1) phase), finite group actions encoded as patterns over indexed expressions (crystal point groups), and other gauges where the transformation is algebraically substitutable. *Not tractable today*: continuous Lie group actions

on subspaces (e.g., $U(d)$ on degenerate-subspace), and data-dependent gauges (e.g. degenerate-subspace rotation that only acts at degenerate ω). For those, the symbolic layer falls back to the Level 1 named-gauge declarations.

Crystal symmetry as a symbolic declaration. Crystal symmetry at the symbolic layer level is purely a *declaration*: a `SymmetryGroup` carries a Hermann-Mauguin / Schoenflies name (`Oh`, `D6h`, `Ci`, ...) and an order $|G|$. No element data — no rotation matrices, no translation vectors, no group-element enumeration — lives in the symbolic layer.

The concrete handling — extracting symmetry from a structure, applying group operations to FC tensors, building irreducible-BZ reductions — belongs to the materials codes (phonopy, kaldo, ShengBTE, ...), typically via `spglib`. The symbolic layer’s role is just to *name* the group so that operations can declare which symmetry they assume as a parameterized identity (`compute_force_constants[order=2, symmetry=Oh]` differs from [..., symmetry=C1]), so adapter specs can declare which group a particular run assumed, and so cross-code comparison can refuse to compare two materializations whose declared symmetry groups disagree.

This is the same pattern `Potential` follows: declared symbolically as a labelled source state without the symbolic layer modeling its functional form. “How $\Phi^{(2)}$ is computed under crystal symmetry” is a code concern; “which symmetry group was assumed” is a symbolic-level declaration.

4.8 Materialization functor

A materialization functor $\mathcal{F}$ is associated with a particular code (kaldo, phono3py, ShengBTE). Given a symbolic workflow W and a discretization scheme Σ ,

$$\mathcal{F}(W, \Sigma) \tag{3}$$

produces a directed graph of materializations whose vertices are in one-to-one correspondence with a subset $V_{\mathcal{F}} \subseteq V(W)$ and whose edges are the computational realizations of the symbolic operations restricted to $V_{\mathcal{F}}$. Different codes implement different functors over the same symbolic workflow; their materialized graphs are aligned by their shared symbolic template.

4.9 Total error

The total error on the materialization m of a symbolic state s is

$$\epsilon_{\text{total}}(m) = \text{compose}(\epsilon_{\text{approx}}(s), \epsilon_{\text{disc}}(m)). \tag{4}$$

The composition rule depends on the operation type but is in all cases derivable from the typed metadata. The two contributions $\epsilon_{\text{approx}}(s)$ and $\epsilon_{\text{disc}}(m)$ are recovered separately, so a result that is converged in discretization but uncertain in approximation is distinguishable from one that is fully approximated but undersampled.

5 Lean-compatibility disciplines

“Lean-compatible” is structural rather than syntactic. The symbolic layer’s Python implementation will not be Lean code, but it should be *shaped* so that the Lean version is a transliteration rather than a redesign. We commit to three disciplines that protect this option.

1. **Closed unions for physics types.** The registry of physics types (`Potential`, `ForceConstants`, `Dispersion`, `ScatteringRates`, ...) is exhaustive and closed—implemented in Python via sealed enums or tagged unions, not via open class inheritance. Lean represents these as inductive types with a fixed list of constructors; closed unions in Python preserve this structure exactly. Open inheritance would require redesign at the Lean stage.

2. **No physics invariants encoded in Python types.** Properties such as “a force-constant tensor is symmetric under the appropriate index exchanges” or “the eigenvectors of a dynamical matrix are orthonormal” are documented as informal invariants on each type but *not* enforced by the Python type system. In Lean these invariants become formal proof obligations attached to the corresponding types. Trying to encode them in Python (via runtime assertions or pydantic validators) creates a layer of checks that does not transliterate and risks coupling the implementation to Python-specific idioms.
3. **Operation parameters always recorded in output state metadata.** When an operation carries parameters that affect physics (§4.2), Python’s type system will not enforce the dependence of the output type on those parameters, but the output state’s metadata must still record them. In Lean these parameters lift to type-level indices, and the type-level dependence is enforced formally; in Python they are runtime metadata that downstream operations and provenance machinery treat as part of the operation’s identity.

These disciplines are inexpensive to follow from day one and prohibitive to retrofit. They keep the transliteration cost low without committing the symbolic layer to Lean tooling we do not yet need.

6 First Demonstration: Lattice Thermal Transport

We instantiate the symbolic layer on the canonical workflow for lattice thermal conductivity κ , computed from second- and third-order interatomic force constants via the linearized Boltzmann transport equation.

6.1 The symbolic DAG

The symbolic DAG has 14 nodes and 13 edges. Two source nodes (**Potential**, **Temperature**) are produced by nullary `provide_*` operations; the **MeanFreeDisplacement** and **ThermalConductivity** states are each split into two parameterized variants by the upstream `bte_solver` choice (one gauge-invariant, one not).

Nodes (Observable / HiddenState):

- **Potential (O)**: the symbolic root, stubbed in Phase 1;
- **Temperature (O)**: scalar source T ;
- **ForceConstants[order=2] (O)**, **[order=3] (O)**: real-space interatomic force constants $\Phi^{(n)}$;
- **DynamicalMatrix (O)**: Bloch sum $D_{ij}(\mathbf{q})$;
- **Frequency (O)**: $\omega(\mathbf{q}, \nu)$, eigenvalues (gauge-invariant);
- **Eigenvectors (H)**: $e(\mathbf{q}, \nu)$, phase + degenerate-subspace freedom;
- **GroupVelocity (H)**: $v^\alpha(\mathbf{q}, \nu)$, inherits eigenvector freedom at degenerate ω ;
- **HeatCapacity (O)**: $c(\mathbf{q}, \nu, T) = (\hbar\omega)^2 / (4k_B T^2 \sinh^2(\hbar\omega/2k_B T))$;
- **Linewidth (H)**: per-mode $\Gamma(\mathbf{q}, \nu, T)$, BZ-summation redistribution;
- **MeanFreeDisplacement[bte_solver=rta] (H)**: RTA $F = v/(2\Gamma)$, inherits Linewidth’s looseness;
- **MeanFreeDisplacement[bte_solver=direct_inverse] (O)**: full LBTE solution, gauge-invariant;

- `ThermalConductivity[bte_solver=rta]` (**H**): the $1/\Gamma$ non-linearity propagates Linewidth's looseness into κ_{RTA} ;
- `ThermalConductivity[bte_solver=direct_inverse]` (**O**): LBTE off-diagonals preserve gauge-invariance.

Edges. Verb-headed names; each carries a sympy formula:

- `provide_potential` (identity), `provide_temperature` (identity);
- `compute_force_constants[order=2]`, `[order=3]` (stubbed): $\Phi^{(n)} = \partial^n V / \partial u^n|_{u=0}$;
- `compute_dynamical_matrix`: Bloch sum $D_{ij}(\mathbf{q}) = \frac{1}{\sqrt{M_i M_j}} \sum_R \Phi_{ij}^{(2)}(R) e^{i\mathbf{q} \cdot \mathbf{R}}$;
- `compute_dispersion` (multi-output): eigenvalue equation $\sum_j D_{ij}(\mathbf{q}) e_{j,q,\nu} = \omega_{q,\nu}^2 e_{i,q,\nu}$;
- `compute_group_velocity`: Hellmann–Feynman $v_{q,\nu}^\alpha = \frac{1}{2\omega_{q,\nu}} e_{q,\nu}^\dagger \partial D / \partial q^\alpha e_{q,\nu}$;
- `compute_heat_capacity`: closed form above;
- `compute_linewidth`: triple BZ sum (Fermi's golden rule); algorithmic convention `broadening_param` $\in \{\text{stdev}, \text{halfwidth}\}$;
- `solve_bte[bte_solver=rta]`: closed form $F = v/(2\Gamma)$;
- `solve_bte[bte_solver=direct_inverse]`: implicit linear system $\sum_{q'\nu'} \mathcal{M}_{q\nu,q'\nu'} F_{q'\nu'}^\alpha = c_{q\nu} v_{q\nu}^\alpha$;
- `contract_kappa[bte_solver=rta]` and `[bte_solver=direct_inverse]`: same BZ contraction $\kappa^{\alpha\beta} = \frac{1}{VN_q} \sum c v^\alpha F^\beta$; the two operations differ only in their input and output state types, ensuring κ_{RTA} is typed as a HiddenState and κ_{LBTE} as an Observable.

Edges are exact symbolic operations (Fourier transform, eigendecomposition, mode contraction) or approximating ones (truncation of the Taylor expansion at second or third order, the relaxation-time approximation when chosen). Each approximating edge carries an explicit error formula (Phase 2 capability). The Phase 1 demonstration exercises the path from `ForceConstants` downward; the upstream operations from `Potential` to `ForceConstants` are declared but executed only as opaque adapter-level steps recorded as provenance, not modeled by the framework.

6.2 Materialization functors (codes)

Each adapter is a materialization functor over the symbolic DAG. Phase 1 supplies two:

- **kaldo** (Python). Computes harmonic and anharmonic phonon transport from force constants, supporting iterative BTE, RTA, and the Wigner formulation. Most intermediate states are exposed via Python attributes on the `Phonons` object. `kaldo's Conductivity(method='inverse')` realizes the canonical `bte_solver=direct_inverse`.
- **phono3py** (C/Python). Computes lattice thermal conductivity from third-order force constants. Intermediate quantities are exposed in HDF5 outputs. `run_thermal_conductivity(is_LBTE=True)` realizes the canonical `bte_solver=direct_inverse`.
- **ShengBTE** (Fortran/MPI; deferred). Solves the full linearized BTE with iterative self-consistency. Adapter declaration is a half-day exercise once ShengBTE is included in the runs.

Both factors were derived by the symbolic layer from declared adapter specs, then applied mechanically to the diagnostic data.

The 4% disagreement on κ_{RTA} is the symbolic layer’s structural prediction working as designed: RTA’s approximation breaks κ ’s gauge invariance via the $1/\Gamma$ non-linearity, so the per-mode Linewidth redistribution propagates into κ . Typing κ_{RTA} as a `HiddenState` makes this a non-anomaly; the gauge-invariant κ_{LBTE} agrees to 0.4%.

6.4 Phase 1 scientific capabilities

Phase 1 yields two immediate scientific capabilities, each of which is an artifact the field does not currently produce. A third capability is the long-term goal toward which the symbolic layer is oriented but is deferred to Phase 2.

- **Cross-code reconciliation at canonical intermediate states.** When kaldo and Sheng-BTE disagree on a final κ , the symbolic layer localizes the divergence to the first symbolic state at which the materializations stop agreeing, and attributes it to a specific operation along the provenance. This is a structural diagnosis: the framework identifies where two codes do something different, regardless of the magnitude of the difference.
- **A unified framework for theory-experiment comparison.** The same symbolic DAG accepts experimental measurements as additional materializations of parameter states, with the same alignment machinery as for cross-code comparison. Phase 1 demonstrates the framework on at least one experimental dataset (e.g., a measured $\kappa(T)$ curve for silicon).

Deferred to Phase 2:

- **Decomposed error bounds on κ .** The symbolic layer is designed to eventually produce the central value of κ together with a symbolic decomposition of its total error into the truncation error of the perturbation series and the discretization error of the chosen mesh. The architecture supports this; the implementation of the composition algebra is a research problem reserved for Phase 2.

7 Implementation layout

The Python package `omai` factors into two top-level sub-packages plus one per domain:

- `omai.symbolic` — symbolic layer machinery for the symbolic world. `State` (base class), `Observable` and `HiddenState` (subclasses, with gauge-discipline fields on the latter), `Field`, `Operation`, `Parameter`, `Dimension`, `PhysicsType`, `topological_order`, `validate_dag` (Level 1 discipline enforcement), and `GaugeAction` / `check_invariance` (Level 2 symbolic gauge proofs). Code-agnostic and domain-agnostic.
- `omai.materialization` — the bridge.
 - `units.py`: the `Unit` class, named unit instances, strict same-dimension `conversion_factor`.
 - `adapter.py`: `StateAdapterSpec` and `OperationAdapterSpec`, plus cross-state and cross-operation comparison helpers (`cross_state_unit_factor`, `cross_state_total_factor`, `cross_operation_*`).
 - `instance.py`: the `Materialization` runtime data class and `materialize()` constructor.
 - `compare.py`: `compare()` with the five-status `ComparisonResult`.

Code-agnostic.

- `omai.thermal_transport` — the lattice-thermal-transport domain.
 - `symbolic/nodes.py`: the 14 states as `Observable` or `HiddenState` instances with their fields.
 - `symbolic/edges.py`: the 13 operations with sympy formulas; sympy symbols and `IndexedBase` declarations.
 - `symbolic/gauges.py`: concrete `GaugeAction` instances for the domain (e.g. U(1) phase on Eigenvectors). Mechanically verifies operation formulas are gauge-invariant.
 - `materialized/kaldo.py`, `materialized/phono3py.py`: per-code adapter specs (one file per code), declaring units, conventions, and discretization choices.
 - `visualize.py`: emits `docs/dag.html`, an interactive single-file visualization (Mermaid layered diagram, click-to-details panels, MathJax-rendered formulas) reflecting the current state.

The split mirrors the symbolic layer’s architectural commitment: **abstract** is the symbolic world; **materialization** is the bridge to the numeric world; **thermal_transport** is the first domain instantiated against them. Future domains (electronic transport, optical response, ...) follow the same pattern in their own folder; no change to the symbolic-level packages is required to add one.

8 Open Questions and Deferred Decisions

We list the architectural questions that remain open and the principled grounds on which they were deferred.

8.1 Algebra of error composition (deferred to Phase 2)

The symbolic layer is designed to carry error formulas as symbolic expressions, but Phase 1 does not implement the algebra by which they compose. Composing two errors in independent small parameters ($O(\lambda^4) + O(N^{-2})$) is well-defined; composing two errors in the same parameter requires care; composing constant-offset errors (e.g., basis-set incompleteness) with asymptotic ones is yet another case. The right algebra is itself a research subproblem and will be developed alongside concrete worked examples in Phase 2. The Phase 1 type system reserves the slots required for error formulas (operation metadata, discretization error model, materialization total error) but populates them with placeholders rather than computed compositions.

8.2 When to lift to Lean

We have committed to keeping the symbolic layer Lean-compatible without being Lean-dependent. The question of when (and whether) to actually project the symbolic layer into Lean is deferred until the Python prototype has stabilized. A reasonable trigger for this decision is the moment at which we have enough concrete examples that designing Lean types in isolation becomes counterproductive.

8.3 Provenance equivalence

We have committed to provenance as essential to identity. This means two states with the same type but different histories are formally distinct. In some cases, however, the two histories are *equivalent*: they produce identical physics under a theorem we can prove (e.g., Fourier transform of a real symmetric matrix and direct frequency-domain assembly should yield the same **Dispersion**). The symbolic layer must provide a mechanism for declaring such equivalences, either as explicit axioms or as proven theorems. The mechanism’s design is deferred to Day 2 because we want concrete examples before designing the equivalence calculus.

8.4 Sprint 1 scope

The first concrete deliverable is a Python implementation. We have considered three scopes:

1. *Narrowest*: kaldo only, harmonic sub-DAG (force constants $\rightarrow$ dynamical matrix $\rightarrow$ dispersion). 4–5 states, fastest to ship.
2. *Recommended*: kaldo only, full thermal-conductivity DAG. 10–15 states, end-to-end on silicon. The symbolic layer skeleton plus one full adapter.
3. *Broader*: kaldo plus phono3py, full DAG. Validates the cross-code story earlier but adds adapter work in parallel with framework design.

The recommended scope (option 2) is the smallest scope that is still a credible proof of concept of the framework, and the choice of one adapter (kaldo) lets us validate the symbolic layer design before generalizing. Sprint 2 adds phono3py and ShengBTE adapters, at which point the cross-code demo of §6 becomes real.

8.5 Implementation language

We have committed to Python for Sprint 1. The trade-offs were:

- **Python (chosen)**. All target codes have Python bindings or wrappable interfaces. The Python type system, augmented with `pydantic` or `attrs` and runtime checks, is sufficient for the symbolic layer’s first-cut type discipline. The development speed is dominant, since adapter work is the bulk of the time cost.
- *Typed compiled language (Rust, OCaml, TypeScript)*. Stronger type guarantees from the start. Rejected for Sprint 1 because the foreign-function interface to existing Python codes adds friction proportional to the symbolic layer’s surface, and we want framework iteration to be cheap.
- *Lean*. Considered as a future projection target and deferred. Not the right tool for running scientific computations; reserved as a verification target.

9 Outlook

The symbolic layer is an architectural commitment, not an implementation. The implementation that follows is scoped to a Python skeleton plus adapters for three thermal-transport codes. The artifact that motivates the project—a paper-worthy demonstration of cross-code reconciliation and decomposed error bounds for lattice thermal conductivity—is downstream of the symbolic layer but is the symbolic layer’s first proof-of-value.

Three longer-term directions follow naturally from the design:

- **Verification**. Project the symbolic layer into Lean (via PhysLean or alongside it), allowing approximation theorems to be formally proved rather than asserted. The provenance mechanism makes this projection well-defined: each operation in π corresponds to a Lean lemma whose statement is the operation’s error formula.
- **Theory-experiment integration**. Extend the materialization functor to ingest experimental observables. The symbolic layer’s graph-alignment machinery applies uniformly, so a measured κ becomes another materialization of the abstract κ , and “does the theory match experiment” becomes graph-alignment with one functor being empirical.

- **Grounded LLM agents.** An LLM operating over typed symbolic states (rather than text tokens) constructs workflows by emitting symbolic operations. The framework type-checks each emission, eliminating the drift problem that plagues current text-token agents. The action space is finite, typed, and physically meaningful.

The unifying claim of the design is that scientific computing has been organized around the wrong atomic unit. The instruction-in-an-input-file unit is too low; the natural-language description is too high; the equation-in-a-paper unit is too disconnected from execution. The right unit, at least for materials science, is the symbolic operation between typed physics states. Once that commitment is made, much of the rest of the architecture writes itself.

This document records the architectural state of the openmaterials-ai project as of May 11, 2026. It is intended as the working reference for the project; subsequent revisions should update the relevant sections rather than appending changelogs.